

Formal Verification: Ray Linear Chain Driver

Zach Kelling, Lux Industries

December 2025

Abstract

We formalize Ray, the linear chain consensus driver that processes items sequentially from a source and commits decided items to a sink in order. We prove that decisions advance a prefix frontier with no gaps and the decided count is monotonically non-decreasing.

1 Introduction

Ray is the linear consensus driver that wraps Wave for sequential block processing. It processes items from a Source, runs Wave voting on each, and commits to a Sink in strict order. The no-gaps invariant ensures that decided items form a contiguous prefix of the source order.

2 Definitions

Definition 1 (RayState). *State tracking source order, decided count (prefix length), and per-item decisions.*

3 Theorems and Axioms

Theorem 2 (decideNext_marks_item). *The `decideNext` function marks exactly the next item as decided.*

Theorem 3 (decideNext_advances). *`decideNext` advances the frontier by exactly 1 when items remain.*

Theorem 4 (init_nothing_decided). *Initial state has decided count 0.*

Theorem 5 (no_gaps). *After a step, the decided count is either $s.decided + 1$ or $s.decided$ (no gaps).*

Theorem 6 (decided_monotone). *The decided count never decreases: $(decideNext\ s\ d).decided \geq s.decided$.*

Proof. By `omega` on both branches of the match. □

4 Lean Source

```
/-
Ray Protocol Formal Model

Models protocol/ray/ray.go -- linear chain consensus driver.

Ray pulls pending items from Source, runs Wave voting on each,
and commits decided items to Sink in order.

Key invariant: items are decided in a total order consistent
with the Source ordering. No reordering. No gaps.

Maps to:
- ray.go: Source.NextPending, Wave.Tick, Sink.Decide
- ray.go: Driver processes items sequentially from Source

Properties:
- Total order: decided items form a prefix of source order
- No gaps: if item at index i is decided, all items at index < i are decided
- Monotonic: decided set only grows
-/

import Mathlib.Data.Nat.Defs
import Mathlib.Data.List.Basic
import Mathlib.Tactic

namespace Protocol.Ray

/-- Item identifier -/
abbrev ItemId := Nat

/-- Decision for an item -/
inductive Decision where
| pending : Decision
| commit  : Decision
| skip    : Decision
deriving DecidableEq

/-- Ray state: tracks the decision frontier -/
structure RayState where
sourceOrder : List ItemId      -- items in source order
decided      : Nat              -- number of items decided (prefix length)
decisions    : ItemId → Decision

/-- Initial state -/
def init (items : List ItemId) : RayState :=
{ sourceOrder := items
, decided := 0
, decisions := fun _ => Decision.pending }

/-- Step: decide the next pending item -/
def decideNext (s : RayState) (d : Decision) (hd : d == Decision.pending) :
RayState :=
```

```

match s.sourceOrder.get? s.decided with
| none => s -- no more items
| some item =>
  { s with
    decided := s.decided + 1
    decisions := fun id => if id = item then d else s.decisions id }

/-- decideNext marks exactly the next item as decided -/
theorem decideNext_marks_item (s : RayState) (d : Decision) (hd : d Decision
.pending)
(item : ItemId) (h_get : s.sourceOrder.get? s.decided = some item) :
(decideNext s d hd).decisions item = d := by
simp [decideNext, h_get]

/-- decideNext advances the frontier by exactly 1 when items remain -/
theorem decideNext_advances (s : RayState) (d : Decision) (hd : d Decision.
.pending)
(item : ItemId) (h_get : s.sourceOrder.get? s.decided = some item) :
(decideNext s d hd).decided = s.decided + 1 := by
simp [decideNext, h_get]

/-- Initial state has nothing decided -/
theorem init_nothing_decided (items : List ItemId) :
(init items).decided = 0 := rfl

/-- No gaps: if item i is decided, item i-1 is decided -/
theorem no_gaps (s s' : RayState) (d : Decision) (hd : d Decision.pending)
(hs : s' = decideNext s d hd) :
s'.decided = s.decided + 1 s'.decided = s.decided := by
simp [decideNext] at hs
split at hs <;> simp_all

/-- Monotonic: decided count never decreases -/
theorem decided_monotone (s : RayState) (d : Decision) (hd : d Decision.
.pending) :
(decideNext s d hd).decided s.decided := by
simp [decideNext]
split <;> omega

end Protocol.Ray

```

5 References

Source code: <https://github.com/luxfi/proofs/blob/main/lean/Protocol/Ray.lean>

White papers: <https://papers.lux.network>

Related white papers:

- lux-consensus.tex